

2024-10-22-Regression

Generated by Scribe.AI

December 31, 2024

Slide 1

Content

Regression

Given $\{x_1, x_2, \dots, x_m\}$ $x_i \in \mathbb{R}$

$$(1) \quad \min_x \frac{1}{m} \sum_{i=1}^m (x - x_i)^2 \implies x = \frac{x_1 + \dots + x_m}{m}$$

L^2 -error (mean)

$$(2) \quad \min_x \frac{1}{m} \sum_{i=1}^m |x - x_i| \implies x = \text{median}$$

L^1 -error

$$(3) \quad \min_x \left(\max_i |x - x_i| \right) \implies x = \frac{\min x_i + \max x_i}{2}$$

L^∞ -error "mid-range"

Description

Slide 1 of 22 introduces the concept of regression in the context of a linear programming course. The slide presents three different regression methods, each minimizing a different type of error: L^2 (least squares), L^1 (least absolute deviations), and L^∞ (minimax). The given data is a set of m real numbers $\{x_1, x_2, \dots, x_m\}$. For each error metric, the slide shows the minimization problem and its solution. The L^2 error minimization leads to the mean of the data as the solution, the L^1 error minimization leads to the median, and the L^∞ error minimization leads to the mid-range (average of the minimum and maximum values). The error metrics (L^2 , L^1 , L^∞) are written in red, highlighting their importance. The context of linear programming is relevant because these regression methods can be formulated as linear programming problems, especially the L^1 regression, which can be expressed as a linear program. The slide serves as an introduction to different regression techniques before potentially delving into their linear programming formulations later in the course.

Figures

Regression

Given $\{x_1, x_2, \dots, x_m\}$

(1) $\min_x \frac{1}{m} \sum_{i=1}^m (y_i - x)^2$
 L^2 -error

(2) $\min_x \frac{1}{m} \sum_{i=1}^m |y_i - x|$
 L^1 -error

(3) $\min_x \max_i |y_i - x|$
 L^∞ -error

(a) Handwritten notes showing different regression methods and their corresponding error metrics. The title "Regression" is underlined in red. The given data set $\{x_1, x_2, \dots, x_m\}$ is defined, with each x_i belonging to the set of real numbers $\mathbb{R}$. Three different minimization problems are presented, each corresponding to a different error metric: L^2 -error (least squares), L^1 -error (least absolute deviations), and L^∞ -error (minimax). The solutions for each minimization problem are given: mean for L^2 , median for L^1 , and mid-range for L^∞ . The error metrics are labeled in red.

Slide 2

Content

Given attributes $\{a_1, a_2, \dots, a_n\}, b$

Find parameters $x_1, x_2, \dots, x_n$ s.t.

$$\underline{b = a_1x_1 + a_2x_2 + \dots + a_nx_n}$$

$$(b = a_1x_1 + a_2x_2 + \dots + a_nx_n + \epsilon) \xrightarrow{\text{error}}$$

Description

Slide 2 of 22 introduces a linear regression problem in the context of a linear programming course. Given a set of attributes $\{a_1, a_2, \dots, a_n\}$ and a target value b , the goal is to find parameters $x_1, x_2, \dots, x_n$ that satisfy the linear equation $b = a_1x_1 + a_2x_2 + \dots + a_nx_n$. The underlined equation represents the ideal case. However, the slide also shows a more realistic scenario where an error term ϵ is introduced, representing the difference between the predicted value ($a_1x_1 + a_2x_2 + \dots + a_nx_n$) and the actual value b . The red arrow and label "error" highlight this discrepancy. The context of linear programming is relevant because finding the optimal parameters x_i that minimize the error (e.g., using least squares or least absolute deviations) can be formulated as a linear programming problem. This slide sets the stage for later discussions on how linear programming techniques can be used to solve regression problems.

Figures

Given attributes $\{a_i\}$
 Find parameters x_i
 $b = a_1x_1 + a_2x_2 + \dots + a_nx_n$

(a) Handwritten notes showing a linear equation and an error term. The given attributes are $\{a_1, a_2, \dots, a_n\}$ and b . The goal is to find parameters $x_1, x_2, \dots, x_n$ such that $b = a_1x_1 + a_2x_2 + \dots + a_nx_n$. A red underline is under the equation. Below that, an error term ϵ is added to the equation, and a red arrow points to it, labeled "error".

Slide 3

Content

Given data points (individuals)

$$b_i, \sum_{j=1}^n a_{ij}x_j, \quad i = 1, \dots, m$$

Find $x_1, x_2, \dots, x_n$ s.t.

$$b_i = \sum_{j=1}^n a_{ij}x_j, \quad i = 1, \dots, m$$

$$b_i = \sum_{j=1}^n a_{ij}x_j + \epsilon_i, \quad i = 1, \dots, m$$

$$\underline{b = AX + \epsilon}$$

Description

Slide 3 of 22 extends the linear regression problem introduced in the previous slide. It formalizes the problem using matrix notation. Given m data points, each represented by a target value b_i and a set of n attributes a_{ij} (where i indexes the data point and j indexes the attribute), the goal is to find the parameters x_j that

best fit the linear model $b_i = \sum_{j=1}^n a_{ij}x_j$. The first equation represents the ideal, error-free case. The second

equation introduces an error term ϵ_i for each data point, acknowledging that the linear model may not perfectly fit the data. The red arrow pointing to ϵ_i and labeled "error" emphasizes this. Finally, the entire system is concisely represented by the matrix equation $\underline{b = AX + \epsilon}$, where b is an $m \times 1$ vector of target values, A is an $m \times n$ matrix of attributes, X is an $n \times 1$ vector of parameters, and ϵ is an $m \times 1$ vector of errors. The underlined equation highlights the compact matrix representation of the problem. The context of linear programming is relevant because finding the optimal parameters X that minimize the error (e.g., using least squares or least absolute deviations) can be formulated as a linear programming problem. This slide bridges the gap between the previous slide's conceptual introduction and the upcoming formulation of the regression problem as a linear program.

Figures

Given data points
 b_i and $a_{ij} \sum_{j=1}^n$
 Find $x_1, x_2, \dots$

(a) Handwritten notes showing a linear equation and an error term. The given data points are b_i , and

$\sum_{j=1}^n a_{ij} x_j$, where $i = 1, \dots, m$.

The goal is to find parameters $x_1, x_2, \dots, x_n$ such that

$$b_i = \sum_{j=1}^n a_{ij} x_j.$$

Below that, an error term ϵ_i is added to the equation, and a red arrow points to it, labeled "error". Finally, a matrix equation $b = AX + \epsilon$ is written, with a red underline.

Slide 4

Content

L²-Regression (Least Square)

$$\min_{(x_1, \dots, x_n)} \sum_i \left[\left(b_i - \sum_j a_{ij} x_j \right)^2 \right]$$

$$\min_X \|b - AX\|_2^2$$

$AX = b$ (might not be solvable)

$A^T A \hat{X} = A^T b$ (normal equation, always solvable)

$\hat{X} = (A^T A)^{-1} A^T b$ (if $(A^T A)^{-1}$ exists)

Description

Slide 4 of 22 presents the least squares regression problem in the context of a linear programming course. The

slide begins by defining the problem as minimizing the sum of squared errors: $\min_{(x_1, \dots, x_n)} \sum_i \left[\left(b_i - \sum_j a_{ij} x_j \right)^2 \right]$,

where b_i represents the observed values, a_{ij} represents the attributes, and x_j are the parameters to be estimated. This is equivalently expressed in matrix notation as $\min_X \|b - AX\|_2^2$, where b is a vector of observed values, A is a matrix of attributes, and X is a vector of parameters. The slide then shows that the system of equations $AX = b$ might not be solvable, leading to the derivation of the normal equation $A^T A \hat{X} = A^T b$, which always has a solution if the matrix $A^T A$ is invertible. The solution is then given as $\hat{X} = (A^T A)^{-1} A^T b$, provided that the inverse exists. The parenthetical comments highlight the solvability conditions. The context of linear programming is relevant because the least squares problem can be formulated as a quadratic programming problem, a close relative of linear programming. The slide lays the groundwork for later discussions on how linear programming techniques can be applied to solve or approximate solutions to least squares problems, especially in cases where the matrix $A^T A$ is not invertible or ill-conditioned.

Figures

L²-Regression (Least

$$\min_{(x_1, \dots, x_n)} \sum_i |e_i|$$

$$\min \|b - AX\|$$

(a) Handwritten notes showing the least squares regression problem and its solution. The title "L²-Regression (Least Square)" is underlined in red. The problem is formulated as minimizing the sum of squared errors, which is equivalent to minimizing the L2 norm of the residual vector $b - AX$. The normal equation $A^T A \hat{X} = A^T b$ is derived, which always has a solution if $(A^T A)^{-1}$ exists. The solution $\hat{X} = (A^T A)^{-1} A^T b$ is given. The parenthetical comments explain the solvability of the equations.

Slide 5

Content

L¹-regression

$$\begin{aligned} & \min_{(x_1, \dots, x_n)} \sum_i \left| b_i - \sum_j a_{ij} x_j \right| \\ & \min_X \|b - AX\|_1 \\ & \min_i \sum_{i=1}^m t_i \\ & s.t. \quad -t_i \leq b_i - \sum_j a_{ij} x_j \leq t_i \end{aligned}$$

Description

Slide 5 of 22 introduces L1 regression, a type of regression analysis that minimizes the sum of the absolute differences between the observed and predicted values. The slide first presents the problem as minimizing the L1 norm of the residual vector: $\min_X \|b - AX\|_1$, where b is the vector of observed values, A is the matrix of attributes, and X is the vector of parameters to be estimated. This is equivalent to minimizing the sum of absolute errors: $\min_{(x_1, \dots, x_n)} \sum_i |b_i - \sum_j a_{ij} x_j|$. The slide then shows how this problem can be reformulated as a linear program by introducing slack variables t_i for each data point. The objective function becomes $\min_i \sum_{i=1}^m t_i$, and the constraints are $-t_i \leq b_i - \sum_j a_{ij} x_j \leq t_i$, which ensures that each t_i is greater than or equal to the absolute error $|b_i - \sum_j a_{ij} x_j|$. This reformulation is crucial because it allows the use of linear programming techniques to solve the L1 regression problem. The arrow points to the constraint that is derived from the absolute value. The context of the course (Linear Programming) is essential here, as the slide demonstrates how a seemingly non-linear problem (L1 regression) can be transformed into a linear program, making it solvable using linear programming algorithms.

Figures

$$\min_{(x_1, \dots, x_n)} \sum_i |b_i - \dots|$$

$$\min_X \|b - AX\|_1$$

(a) Handwritten notes showing the L1 regression problem formulated as a linear program. The objective function is to minimize the sum of absolute errors, which is equivalent to minimizing the L1 norm of the residual vector $b - AX$. The problem is then reformulated as a linear program by introducing slack variables t_i to represent the absolute errors. The constraints ensure that each t_i is greater than or equal to the absolute error.

Slide 6

Content

L^∞ -Regression

$$\begin{aligned} & \min \left(\max_i \left| b_i - \sum_j a_{ij} x_j \right| \right) \\ & \min_X \|b - AX\|_\infty \\ & \min \delta \\ \text{s.t.} \quad & -\delta \leq b_i - \sum_j a_{ij} x_j \leq \delta \end{aligned}$$

Description

Slide 6 of 22 introduces L regression, which aims to minimize the maximum absolute error. The slide begins by defining the problem as minimizing the maximum absolute difference between observed and predicted

values: $\min \left(\max_i |b_i - \sum_j a_{ij} x_j| \right)$, which is equivalent to minimizing the infinity norm of the residual vector:

$\min_X \|b - AX\|_\infty$. The slide then shows how this problem can be reformulated as a linear program. A slack variable δ is introduced to represent the maximum absolute error. The objective function becomes $\min \delta$, and the constraints are $-\delta \leq b_i - \sum_j a_{ij} x_j \leq \delta$ for all i , ensuring that all absolute errors are less than or

equal to δ . The arrow points to the constraint derived from the absolute value condition. This reformulation is key because it allows the use of linear programming algorithms to solve the L regression problem. The context of the course (Linear Programming) is crucial, as the slide demonstrates how a minimax problem (L regression) can be transformed into a linear program.

Figures

$$\min (\max_i |b_i -$$

$$\min_X \|b - AX\|_1$$

(a) Handwritten notes showing the L regression problem formulated as a linear program. The objective function is to minimize the maximum absolute error, which is equivalent to minimizing the L norm of the residual vector $b - AX$. The problem is then reformulated as a linear program by introducing a slack variable to represent the maximum absolute error. The constraints ensure that each absolute error is less than or equal to .

Slide 7

Content

Binary Classification given labelled points
 (x_i^1, x_i^2)

Description

Slide 7 of 22 introduces a binary classification problem in the context of linear programming. The slide presents a scatter plot of data points, visually separated into two classes. Blue circles with "+" symbols represent one class, while red circles with "-" symbols represent the other. Each point is labeled with coordinates (x_i^1, x_i^2) , indicating its position in a two-dimensional feature space. The title "Binary Classification" is underlined, emphasizing the nature of the problem. The phrase "given labelled points" clarifies that the class labels for each data point are known. This visual representation sets the stage for subsequent slides that might discuss how linear programming techniques can be used to find an optimal separating hyperplane (a line in this 2D case) to classify new, unseen data points. The context of linear programming is relevant because finding the optimal separating hyperplane can be formulated as a linear programming problem, particularly if one considers methods like Support Vector Machines (SVMs), which are closely related to linear programming.

Figures

Binary Classification



(a) A diagram showing a binary classification problem. Blue circles with "+" signs represent one class, and red circles with "-" signs represent the other class. The points are labeled (x_i^1, x_i^2) .

Slide 8

Content

Binary Classification given labelled points
 (x_i^1, x_i^2)

Description

Slide 8 of 22 illustrates a binary classification problem. The slide shows a scatter plot with two classes of data points. Blue circles marked with "+" represent one class, and red circles marked with "-" represent the other. Each point is labeled with coordinates (x_i^1, x_i^2) . A black line is drawn to separate the two classes, labeled as "separating line (hyperplane)". The title "Binary Classification given labelled points" indicates that the problem involves classifying data points into two categories based on their known labels. The context of the course (Linear Programming) is important because finding the optimal separating hyperplane can be formulated as a linear programming problem. This is a visual representation of the problem that will likely be followed by slides explaining how linear programming can be used to solve it (e.g., using Support Vector Machines).

Figures

Binary Classification



(a) A diagram showing a binary classification problem. Blue circles with "+" signs represent one class, and red circles with "-" signs represent the other class. The points are labeled (x_i^1, x_i^2) . A black line separates the two classes, labeled "separating line (hyperplane)".

Slide 9

Content

maximize (minimum distance)

Description

Slide 9 of 22 presents a geometric illustration of a linear programming problem related to maximizing the minimum distance between two sets of points. The slide shows two non-convex regions, one colored blue and the other red, representing two classes of data points. A black line separates the two regions. Double-headed arrows indicate the shortest distance between the two regions, which is perpendicular to the separating line. The objective is to maximize this minimum distance. This is a visual representation of a problem that can be formulated as a linear program, possibly related to Support Vector Machines (SVMs) or other classification methods where finding the optimal separating hyperplane involves maximizing the margin (the minimum distance between the hyperplane and the closest data points of each class). The context of the course (Linear Programming) is crucial because the slide illustrates a geometric interpretation of an optimization problem that can be solved using linear programming techniques.

Figures



(a) A diagram showing two irregularly shaped blobs, one blue and one red, separated by a black line. Double-headed arrows point to the minimum distance between the blobs, which is perpendicular to the separating line. The text "maximize (minimum distance)" is written above the diagram.

Slide 10

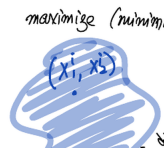
Content

maximize (minimum distance)
 $a_1x_1 + a_2x_2 - b = 0$

Description

Slide 10 of 22 shows a geometric interpretation of a linear programming problem in the context of binary classification. Two irregularly shaped clusters of points, one blue and one red, represent two classes of data. A black line separates the two clusters. Double-headed arrows indicate the minimum distance between the two clusters, measured perpendicularly to the separating line. The objective is to maximize this minimum distance, which is a common goal in Support Vector Machine (SVM) classification. The labels (x_i^1, x_i^2) are written within each cluster, representing the coordinates of data points within each class. The equation $a_1x_1 + a_2x_2 - b = 0$ represents the separating hyperplane (a line in this 2D case). The context of linear programming is crucial because finding the optimal separating hyperplane that maximizes the minimum distance can be formulated as a linear programming problem. This slide visually represents the optimization problem that will be solved using linear programming techniques in subsequent slides.

Figures



(a) A diagram showing two irregularly shaped blobs, one blue and one red, separated by a black line. Double-headed arrows point to the minimum distance between the blobs, which is perpendicular to the separating line. The blue blob is labeled (x_i^1, x_i^2) in blue ink, and the red blob is labeled (x_i^1, x_i^2) in red ink. The text "maximize (minimum distance)" is written above the diagram. The equation $a_1x_1 + a_2x_2 - b = 0$ is written to the upper right of the diagram, representing the equation of the separating line.

Slide 11

Content

$$\begin{aligned} E_i &= a_1x_i^1 + a_2x_i^2 - b \\ (x_i^1, x_i^2) &+ \\ \mathcal{E}_i &= b - (a_1x_i^1 + a_2x_i^2) \\ a_1x_i^1 + a_2x_i^2 - b &> 0 \\ a_1x_i^1 + a_2x_i^2 - b &= 0 \\ a_1x_i^1 + a_2x_i^2 - b &< 0 \end{aligned}$$

Description

Slide 11 of 22 illustrates the geometric interpretation of the distance from data points to a separating hyperplane in a binary classification problem. The equation $\mathcal{E}_i = a_1x_i^1 + a_2x_i^2 - b$ represents the signed distance of a point (x_i^1, x_i^2) from the hyperplane $a_1x_1 + a_2x_2 - b = 0$. A positive value indicates the point is above the hyperplane, a negative value indicates it's below, and zero means it lies on the hyperplane. The diagram shows two points, one above and one below the hyperplane, with their respective signed distances $\mathcal{E}_i$ calculated. The perpendicular distances from the points to the hyperplane are visually represented. The context of linear programming is relevant because finding the optimal separating hyperplane often involves maximizing the margin (minimum distance to the closest points), which can be formulated as a linear programming problem. This slide provides a visual and algebraic representation of the distance calculation, crucial for understanding the optimization problem in subsequent slides.

Figures

$$\mathcal{E}_i = a_1 x_i^1 + a_2 x_i^2 - b$$



(a) A diagram showing a line $a_1 x_1 + a_2 x_2 - b = 0$ separating two classes of points. A blue point labeled (x_i^1, x_i^2) with a "+" sign is above the line, and a red point labeled (x_i^1, x_i^2) with a "-" sign is below the line. Perpendicular lines are drawn from each point to the separating line. The distances from the points to the line are labeled $\mathcal{E}_i$ and $\mathcal{E}_i$, respectively. The equations $a_1 x_i^1 + a_2 x_i^2 - b > 0$, $a_1 x_i^1 + a_2 x_i^2 - b = 0$, and $a_1 x_i^1 + a_2 x_i^2 - b < 0$ are written near the line to indicate the regions where the expression is positive, zero, and negative, respectively.

Slide 12

Content

Distance of $\{\oplus\}$ from $a_1x_1 + a_2x_2 - b$
 $= \min_i (a_1x_i^1 + a_2x_i^2 - b)$

Distance of $\{\ominus\}$ from $a_1x_1 + a_2x_2 - b$
 $= \min_i (b - (a_1x_i^1 + a_2x_i^2))$

Description

Slide 12 of 22 presents formulas for calculating the minimum distances from two sets of points to a separating hyperplane. The first formula calculates the minimum distance from a set of points labeled with a plus sign ($\oplus$), representing points above the hyperplane, to the hyperplane $a_1x_1 + a_2x_2 - b = 0$. The minimum distance is computed as $\min_i (a_1x_i^1 + a_2x_i^2 - b)$, where (x_i^1, x_i^2) are the coordinates of the i -th point in the set. The second formula calculates the minimum distance from a set of points labeled with a minus sign ($\ominus$), representing points below the hyperplane, to the same hyperplane. This distance is given by $\min_i (b - (a_1x_i^1 + a_2x_i^2))$. The context of linear programming is crucial because these formulas are used to define the margin in Support Vector Machines (SVMs), a classification method that can be formulated as a linear program. The goal is to find the hyperplane that maximizes the minimum distance to the closest points of each class, thus maximizing the margin.

Figures

$$\begin{aligned} & \text{Distance of } \{ \oplus \} \\ & = \min_{i, \oplus} (a_i x_i) \end{aligned}$$

(a) The image shows two equations. The first equation calculates the minimum distance from a set of points represented by a blue plus sign ($\oplus$) to a line represented by the equation $a_1 x_1 + a_2 x_2 - b = 0$. The second equation calculates the minimum distance from a set of points represented by a red minus sign ($\ominus$) to the same line. The equations use the notation $\min_i$ to indicate that the minimum distance is taken over all points in each set. The points are labeled with superscripts 1 and 2, suggesting they are two-dimensional coordinates.

Slide 13

Content

$$\begin{aligned}
 &\max \quad \delta \\
 &\delta, \mathcal{E}_i, a_1, a_2, b \\
 &\text{s.t.} \\
 &\mathcal{E}_i = a_1 x_i^1 + a_2 x_i^2 - b \quad \text{if Label}(i) = \oplus \\
 &\mathcal{E}_i = b - a_1 x_i^1 - a_2 x_i^2 \quad \text{if Label}(i) = \ominus \\
 &\delta \leq \mathcal{E}_i
 \end{aligned}$$

Description

Slide 13 of 22 formulates the optimization problem for finding the optimal separating hyperplane in a Support Vector Machine (SVM) context as a linear program. The objective is to maximize the margin, represented by δ . The constraints involve the signed distances $\mathcal{E}_i$ of each data point (x_i^1, x_i^2) from the hyperplane $a_1 x_1 + a_2 x_2 - b = 0$. The signed distance is positive if the point is above the hyperplane ($\text{Label}(i) = \oplus$) and negative if it's below ($\text{Label}(i) = \ominus$). The constraint $\delta \leq \mathcal{E}_i$ ensures that the margin δ is less than or equal to the distance of each point to the hyperplane, effectively maximizing the minimum distance to the closest points. The variables to be optimized are δ , a_1 , a_2 , and b , which define the hyperplane and the margin. This slide bridges the gap between the geometric intuition of maximizing the minimum distance (from previous slides) and its formal mathematical representation as a linear programming problem.

Figures

$$\begin{aligned}
 &\max \quad \delta \\
 &\delta, \mathcal{E}_i, a_1, a_2, b \\
 &\text{s.t.} \\
 &\mathcal{E}_i = a_1 x_i^1 + a_2 x_i^2 - b \\
 &\mathcal{E}_i = b - a_1 x_i^1 - a_2 x_i^2
 \end{aligned}$$

(a) No figure present.

Slide 14

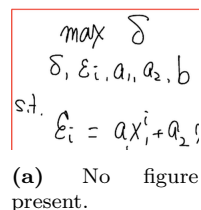
Content

$$\begin{aligned}
 &\max \quad \delta \\
 &\delta, \mathcal{E}_i, a_1, a_2, b \\
 &\text{s.t.} \\
 &\mathcal{E}_i = a_1 x_i^1 + a_2 x_i^2 - b \quad \text{if Label}(i) = \oplus \\
 &\mathcal{E}_i = b - a_1 x_i^1 - a_2 x_i^2 \quad \text{if Label}(i) = \ominus \\
 &\delta \leq \mathcal{E}_i \\
 &-1 \leq a_1 \leq 1, \quad -1 \leq a_2 \leq 1 \quad (\text{normalization})
 \end{aligned}$$

Description

Slide 14 of 22 presents a linear programming formulation for finding the optimal separating hyperplane in a Support Vector Machine (SVM) context. It builds upon the previous slide by adding normalization constraints. The objective is to maximize the margin δ . The constraints are similar to slide 13, defining the signed distances $\mathcal{E}_i$ of data points from the hyperplane $a_1 x_1 + a_2 x_2 - b = 0$. The sign of $\mathcal{E}_i$ depends on the label of the point (positive for $\oplus$, negative for $\ominus$). The constraint $\delta \leq \mathcal{E}_i$ ensures that the margin is less than or equal to the distance of each point to the hyperplane. Crucially, this slide introduces normalization constraints: $-1 \leq a_1 \leq 1$ and $-1 \leq a_2 \leq 1$. These constraints are added to improve the numerical stability and efficiency of the optimization algorithm. The variables to be optimized are δ , a_1 , a_2 , and b . The context of linear programming is essential because this slide presents a complete and practical formulation of the SVM optimization problem as a linear program, ready for solution using standard linear programming techniques.

Figures



Handwritten mathematical formulation of the SVM linear programming problem:

$$\begin{aligned}
 &\max \quad \delta \\
 &\delta, \mathcal{E}_i, a_1, a_2, b \\
 &\text{s.t.} \\
 &\mathcal{E}_i = a_1 x_i^1 + a_2 x_i^2 - b
 \end{aligned}$$

(a) No figure present.

Slide 15

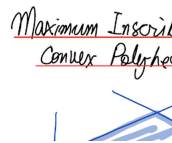
Content

Maximum Inscribed Circle in a
Convex Polyhedron [MG, Sec 2.6]

Description

Slide 15 of 22 discusses the problem of finding the maximum inscribed circle within a convex polyhedron. The title "Maximum Inscribed Circle in a Convex Polyhedron" is underlined, with a reference to a textbook ([MG, Sec 2.6]). The figure depicts a convex polygon (likely a hexagon, though not perfectly drawn) with parallel lines filling its interior, representing a cross-section of a polyhedron. Several lines intersect the polygon, possibly representing supporting hyperplanes or constraints relevant to finding the maximum inscribed circle. The context of linear programming is relevant because finding the maximum inscribed circle within a convex polyhedron can be formulated as a linear programming problem. The problem involves finding the largest circle that can be placed inside the polyhedron without intersecting its boundaries. This is a geometric optimization problem that can be tackled using linear programming techniques.

Figures



(a) The figure shows a convex polygon (it appears to be a hexagon, but it's not perfectly drawn) with parallel lines drawn inside, suggesting the interior of a polyhedron. Several lines intersect the polygon, possibly representing supporting hyperplanes or constraints. The lines are blue, and the polygon is shaded with light blue parallel lines.

Slide 16

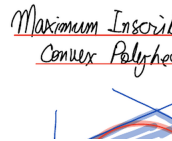
Content

Maximum Inscribed Circle in a
Convex Polyhedron [MG, Sec 2.6]

Description

Slide 16 of 22 introduces the problem of finding the maximum inscribed circle within a convex polyhedron. The title, "Maximum Inscribed Circle in a Convex Polyhedron," is handwritten, with "Convex Polyhedron" underlined. A reference "[MG, Sec 2.6]" is given, suggesting a source for further information. The accompanying hand-drawn figure illustrates a convex polygon (roughly hexagonal) containing an inscribed circle. The polygon is shaded with parallel lines, and several lines intersect it, likely representing constraints or supporting hyperplanes relevant to the optimization problem. The context of linear programming is implied because finding the largest inscribed circle within a convex polyhedron can be formulated as a linear program. The slide transitions from the previous discussion of hyperplanes and margins in SVMs to a related geometric optimization problem solvable using linear programming techniques.

Figures



(a) A hand-drawn figure shows a convex polygon (approximately a hexagon) with a red circle inscribed within it. The polygon is shaded with diagonal blue lines. Several blue lines intersect the polygon, possibly representing supporting hyperplanes or constraints. The drawing is not precise.

Slide 17

Content

$$\underline{L_i a_i^1 x_1 + a_i^2 x_2 - b^i < 0}$$

Description

Slide 17 of 22 illustrates a constraint in the context of finding the maximum inscribed circle within a convex polyhedron. The figure shows a convex polygon (approximately a hexagon) representing a cross-section of a polyhedron. Several lines intersect the polygon, representing the constraints defining the polyhedron. One line, labeled L_i , is highlighted. Below the figure, the inequality $a_i^1 x_1 + a_i^2 x_2 - b^i < 0$ is written and underlined in red. This inequality represents a half-space constraint defining one of the sides of the polyhedron. The variables x_1 and x_2 represent coordinates, and a_i^1 , a_i^2 , and b^i are parameters defining the line L_i . The inequality indicates that the feasible region lies on one side of the line L_i . The context of linear programming is crucial because this slide shows how the geometric problem of finding the maximum inscribed circle can be expressed using linear inequalities, which are fundamental to linear programming formulations. The figure and inequality together represent a single constraint in a system of linear inequalities that define the feasible region for the optimization problem.

Figures



(a) A hand-drawn figure shows a convex polygon (approximately a hexagon) intersected by several lines. One line is highlighted in a pale orange/beige color and labeled L_i . An inequality $a_i^1 x_1 + a_i^2 x_2 - b^i < 0$ is written below the figure, with the inequality underlined in red.

Slide 18

Content

L_i

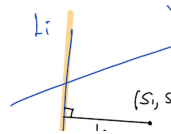
Description

Slide 18 of 22 shows a geometric interpretation of the distance from the center of the inscribed circle to a constraint hyperplane in the context of finding the maximum inscribed circle within a convex polyhedron. The figure depicts a convex polygon (approximately a hexagon) representing a cross-section of the polyhedron. Several blue lines intersect the polygon, representing the constraints defining the polyhedron. One line, L_i , is highlighted in a light orange/beige color. A point, labeled (s_1, s_2) and identified as the "center," represents the center of the inscribed circle. The perpendicular distance from the center to the line L_i is labeled d_i .

The formula $d_i = \frac{|a_i^1 s_1 + a_i^2 s_2 - b^i|}{\sqrt{(a_i^1)^2 + (a_i^2)^2}}$ is given, which calculates this distance. The variables s_1 and s_2 are

the coordinates of the circle's center, and a_i^1 , a_i^2 , and b^i are parameters defining the line L_i . This formula is crucial in formulating the linear program to find the maximum inscribed circle, as it expresses the distance from the center to each constraint as a linear function of the center's coordinates. The context of linear programming is essential because this slide connects the geometric problem to the algebraic formulation needed for solving it using linear programming techniques.

Figures



(a) A hand-drawn figure shows a convex polygon (approximately a hexagon) intersected by several blue lines. One line, labeled L_i , is highlighted in light orange/beige. A small black dot is labeled (s_1, s_2) (center). A short line segment extends from the L_i line to the dot, labeled d_i . A formula is written below the figure: $d_i = \frac{|a_i^1 s_1 + a_i^2 s_2 - b^i|}{\sqrt{(a_i^1)^2 + (a_i^2)^2}}$.

Slide 19

Content

$$\begin{aligned} &\text{maximize } d \\ &(s_1, s_2, d) \\ &\text{s.t. } d \leq d_i = \frac{|a_i^1 s_1 + a_i^2 s_2 - b^i|}{\sqrt{(a_i^1)^2 + (a_i^2)^2}} \\ &a_i^1 s_1 + a_i^2 s_2 - b^i \leq 0 \end{aligned}$$

Description

Slide 19 of 22 presents the linear programming formulation for finding the maximum inscribed circle within a convex polyhedron. The objective is to maximize the radius d of the inscribed circle. The variables are s_1 and s_2 (coordinates of the circle's center) and d (the radius). The constraints ensure that the circle remains inside the polyhedron. Each constraint $d \leq d_i$ limits the radius d to be less than or equal to the distance d_i from the center (s_1, s_2) to the i -th constraint hyperplane, calculated using the distance formula. The inequality $a_i^1 s_1 + a_i^2 s_2 - b^i \leq 0$ ensures that the center lies within the feasible region defined by the hyperplane. The slide builds upon the previous slides by translating the geometric problem into an algebraic linear program, ready to be solved using linear programming techniques. The context of linear programming is crucial here, as the slide shows how a geometric optimization problem can be formulated as a linear program.

Figures

$$\begin{aligned} &\text{maximize } d \\ &(s_1, s_2, d) \\ &\text{s.t. } d \leq d_i \end{aligned}$$

(a) A handwritten slide shows a linear program formulation. The objective function is to maximize d , where d is a variable. The variables are s_1 , s_2 , and d . The constraints are that d must be less than or equal to d_i , where d_i is calculated using the formula shown. There is also a constraint that $a_i^1 s_1 + a_i^2 s_2 - b^i \leq 0$.

Slide 20

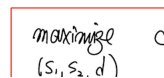
Content

maximize d
 (s_1, s_2, d)
 s.t. $d \leq \frac{b^i - a_i^1 s_1 - a_i^2 s_2}{\sqrt{(a_i^1)^2 + (a_i^2)^2}}$
 $a_i^1 s_1 + a_i^2 s_2 - b^i \leq 0$
 redundant

Description

Slide 20 of 22 refines the linear program formulation for finding the maximum inscribed circle. The objective remains to maximize the radius d . The variables are still s_1 , s_2 (center coordinates), and d (radius). The key change is in the constraints. The previous constraint $a_i^1 s_1 + a_i^2 s_2 - b^i \leq 0$ is now crossed out and labeled "redundant," indicating that it's implied by the other constraint and thus unnecessary. The remaining constraint $d \leq \frac{b^i - a_i^1 s_1 - a_i^2 s_2}{\sqrt{(a_i^1)^2 + (a_i^2)^2}}$ directly relates the radius d to the distance from the center to the i -th constraint hyperplane. This simplification streamlines the linear program, making it more efficient to solve. The context of linear programming is crucial because this slide demonstrates a common step in formulating and refining linear programs: identifying and removing redundant constraints to improve computational efficiency. The simplification makes the linear program more efficient to solve.

Figures



$$\begin{array}{ll} \text{maximize} & d \\ \text{subject to} & (s_1, s_2, d) \end{array}$$

(a) A handwritten slide shows a linear program formulation. The objective function is to maximize d , where d is a variable. The variables are s_1 , s_2 , and d . The constraints are that d must be less than or equal to a fraction involving b^i , a_i^1 , a_i^2 , s_1 , and s_2 . A second constraint is crossed out and labeled "redundant".

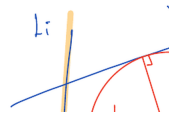
Slide 21

Content

Description

Slide 21 of 22 provides a geometric illustration to support the linear programming formulation for finding the maximum inscribed circle within a convex polygon. The figure shows a circle inscribed within a convex polygon, which is a simplified representation of the higher-dimensional problem. Several lines represent the constraints defining the polygon. The line L_i represents a specific constraint. The distance from the center of the circle to the constraint line L_i is denoted as d_i , while the radius of the circle is d . The right angles indicate that the lines are tangent to the circle at the points of intersection. This visual aids in understanding the relationship between the radius (d) and the distance to each constraint (d_i) used in the linear program formulation from the previous slides. The context of linear programming is crucial here, as the figure provides a geometric interpretation of the constraints in the linear program, showing how the radius of the inscribed circle is related to the distances to the constraint hyperplanes.

Figures



(a) A hand-drawn figure shows a circle inscribed within a convex polygon (approximately a hexagon). The circle is red, and the polygon is defined by several blue lines. A line, labeled L_i , is drawn in light orange/beige, intersecting the circle. A segment is drawn from the center of the circle to the intersection point of L_i and the circle, labeled d_i . Another segment is drawn from the center of the circle to the edge of the circle, labeled d . Right angles are marked where the lines are tangent to the circle.

Slide 22

Content

[MGJ]

8.7 Smallest Balls and Convex Programming

The smallest ball problem. We are given points $p_1, \dots, p_n \in \mathbb{R}^d$, and we want to find a ball of the smallest radius that contains all the points.

Description

Slide 22 of 22 introduces the "Smallest Ball Problem" in the context of convex programming. The problem statement is given: find the smallest radius ball (circle in 2D) that encloses a given set of points $p_1, \dots, p_n$ in $\mathbb{R}^d$ (d -dimensional space). The slide includes a simple hand-drawn illustration in 2D, showing a circle enclosing several points. The figure visually represents the problem's goal: to find the smallest circle that contains all the given points. The notation $p_1, \dots, p_n \in \mathbb{R}^d$ indicates that the points are in a d -dimensional Euclidean space. The context of convex programming is crucial because finding the smallest enclosing ball is a problem that can be formulated and solved using convex programming techniques. The slide concludes the presentation by posing this problem as a final application of the concepts covered in the course.

Figures

[MGJ]

8.7 Smallest Balls and Convex Programming

The smallest ball problem. We are given points $p_1, \dots, p_n \in \mathbb{R}^d$, and we want to find a ball of the smallest radius that contains all the points.

(a) A hand-drawn figure shows a circle encompassing several points. The points are represented by filled black circles, and the circle is drawn to enclose all of them with the smallest possible radius. The circle's center is not explicitly marked.